

Mizar Evo

Readable, AI-Ready, Scalable Formal Mathematics

Eight problems, eight proposals

A discussion with the Bialystok Mizar team, September 2026

presenter notes edition

Mizar Evo project

September 2026

Presenter script:

- Begin with gratitude.
- State that this is a design review with the people best positioned to judge whether the project is still recognizably Mizar.
- Nothing here is frozen; the goal of the visit is to collect objections.

Legacy Mizar exact MML excerpt

```
definition
  struct (1-sorted) addMagma (# carrier -> set, addF -> BinOp of the carrier
    #);
end;
```

Mizar Evo specification example

```
definition
  struct AddMagma where
    field carrier -> set;
    field add -> BinOp of carrier;
  end;

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;      :: renamed view
  end;
end;
```

Presenter script: Say:

- Source: current MML algstr_0.miz, lines 37-40.
- This is the whole talk in one slide: it still reads as Mizar, the mathematics is unchanged, and the things that used to be implicit - the parent link, the field mapping - are now visible, checkable source text.

```

0.1 - A First Look

Legacy Mix legacy-mix example
defunctio
    return [SourceId address 0 source ==> dest, dest ==> source] or dest current
    0;
end;

Mizar Eval mizar-eval example
defunctio
    return address ==> source
    field current ==> dest;
    field use ==> source or current;
end;

defunctio address ==> source from use
    field current from current;
    field use from source;    :: F.M.IZAR.EVAL
end;
end;

```

Key Phrase:

*Preserve Mizar's mathematical vernacular.
Modernize the compiler, verifier, artifact, and publication layers.*

What this talk does not claim:

- Full MML migration is not complete.
- The final language standard is not frozen.
- AI assistance is not a substitute for proof checking.
- The current Mizar system's achievements are the baseline, not the problem.

*Present Mizar's mathematical vernacular.
Modernize the compiler, syntax, artifact and publication layers.*

What this talk does not claim:

- Full MML migration is not complete.
- The final language standard is not frozen.
- AI assistance is not a substitute for proof checking.
- The current Mizar system's achievements are the baseline, not the problem.

Presenter script: Read aloud:

- Full MML migration is not complete.
- The final language standard is not frozen.
- AI assistance is not a substitute for proof checking.

After the example, say which design obligation the syntax makes visible.

Every code example is labeled:

Label	Meaning
exact MML excerpt	verbatim current MML text, with article and line numbers
specification example	from the Mizar Evo language specification
sketch	illustrative; not yet fixed by the specification

Reading rule:

- No EBNF appears in this talk. The language is shown through examples only; `doc/spec/en/` remains the authority for grammar and edge cases.

Every code example is labeled:

Label	Meaning
must: MML excerpt	complete context: MML text, with article and line numbers
specifies example	from the Mizar language specification
sketch	Illustration; not yet fixed by the specification

Reading rule:

- No EBNF appears in this talk. The language is shown through examples only; doc/spec/en/ remains the authority for grammar and edge cases.

Presenter script: Read aloud:

- No EBNF appears in this talk. The language is shown through examples only; doc/spec/en/ remains the authority for grammar and edge cases.

For the table, read the row labels first, then the design reason.

Key Points:

- identify compatibility constraints that are invisible from outside MML work;
- choose migration benchmark articles that are small but representative;
- review the trust boundary for ATP search and kernel checking;
- discuss how Formalized Mathematics should link to a package library;
- collect the objections we have not thought of.

Running question:

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

Key Points:

- identify compatibility constraints that are invisible from outside MML work;
- choose migration benchmark articles that are small but representative;
- review the trust boundary for ATP search and kernel checking;
- discuss how Formalized Mathematics should live in a package library;
- collect the objections we have not thought of.

Running question:

What must Mizar Evo propose so that the Mizar community still recognizes it as Mizar?

Presenter script: Read aloud:

- identify compatibility constraints that are invisible from outside MML work;
- choose migration benchmark articles that are small but representative;
- review the trust boundary for ATP search and kernel checking;

After the example, say which design obligation the syntax makes visible.

Key Points:

- declarative proof text that reads as mathematics;
- soft types, modes, and adjective-rich vocabulary;
- attributes, registrations, and clusters as reusable automation;
- a mature curated library (MML 5.94.1493: 1493 articles);
- a publication culture around formal articles (Formalized Mathematics).

Message:

- These strengths are the design baseline. Every proposal in this talk is judged by whether it protects them.

Mizar Evo

└ Part 1. Why Now

└ 1.1 - What Mizar Got Right

Key Points:

- declarative proof text that reads as mathematics
- soft types, modes, and adjective-rich vocabulary
- attributes, registrations, and clusters as reusable automation
- a mature coded library (MML 5,346,340): 3432 articles
- a publication culture of formal articles (Formalized Mathematics).

Message:

- These strengths are the design baseline. Every proposal in this talk is judged by whether it protects them.

Presenter script: Say:

- Do not lecture the audience about their own system; this frame is one minute of shared ground, not a tutorial.

Read aloud:

- declarative proof text that reads as mathematics;
- soft types, modes, and adjective-rich vocabulary;
- attributes, registrations, and clusters as reusable automation;

Key Points:

- MML has grown to roughly 1500 interdependent articles.
- The unit of dependency, review, and reuse is the whole article.
- Article environments are resolved by tooling, but the resolved dependency surface is not visible in the source a human reviews.
- Whole-library maintenance operations (renames, refactorings, revisions) carry risk that grows with library size.

Message:

- The problem is not that current Mizar is wrong. It is that article-level boundaries were designed for a smaller library.

Key Points:

- MML has grown to roughly 1500 interdependent articles.
- The unit of dependency, review, and reuse is the whole article.
- Article environments are resolved by tooling, but the resolved dependency surface is not visible in the source a human reviews.
- Whole-library maintenance operations (renames, refactorings, revisions) carry risk that grows with library size.

Message:

- The problem is not that current Mizar is wrong. It is that article-level boundaries were designed for a smaller library.

Presenter script: Read aloud:

- MML has grown to roughly 1500 interdependent articles.
- The unit of dependency, review, and reuse is the whole article.
- Article environments are resolved by tooling, but the resolved dependency surface is not visible in the source a human reviews.
- Whole-library maintenance operations (renames, refactorings, revisions) carry risk that grows with library size.
- The problem is not that current Mizar is wrong. It is that article-level boundaries were designed for a smaller library.

Key Points:

- Editors are expected to give instant, partial, resilient feedback.
- Builds are expected to be reproducible from a manifest and lockfile.
- Reuse is expected to work at package granularity with versioning.
- Documentation is expected to be generated, linked, and browsable.

Message:

- These expectations were set by mainstream language ecosystems; new users arrive with them, and formal libraries are judged against them.

Key Points:

- Editors are expected to give instant, partial, resilient feedback.
- Builds are expected to be reproducible from a manifest and lockfile.
- Reuse is expected to work at package granularity with versioning.
- Documentation is expected to be generated, linked, and browsable.

Message:

- These expectations were set by mainstream language ecosystems; new users arrive with them, and formal libraries are judged against them.

Presenter script: Read aloud:

- Editors are expected to give instant, partial, resilient feedback.
- Builds are expected to be reproducible from a manifest and lockfile.
- Reuse is expected to work at package granularity with versioning.
- Documentation is expected to be generated, linked, and browsable.
- These expectations were set by mainstream language ecosystems; new users arrive with them, and formal libraries are judged against them.

Key Points:

- AI agents are already useful for search, explanation, and repair.
- They need bounded, structured, source-anchored context - not a dump of the whole library.
- Their output must never define proof truth; verification must stay independent of the strength of the assistant.
- Readable source is an advantage here: stable local text patterns are what AI edits and retrieval work best on.

Message:

- Mizar's readability is not a nostalgic asset. It is exactly what makes safe AI assistance possible.

Key Points:

- AI agents are already useful for search, explanation, and repair.
- They need bounded, structured, source-anchored context - not a dump of the whole library.
- Their output must never define proof truth; verification must stay independent of the strength of the assistant.
- Readable source is an advantage here: stable local text patterns are what AI edits and retrieval work best on.

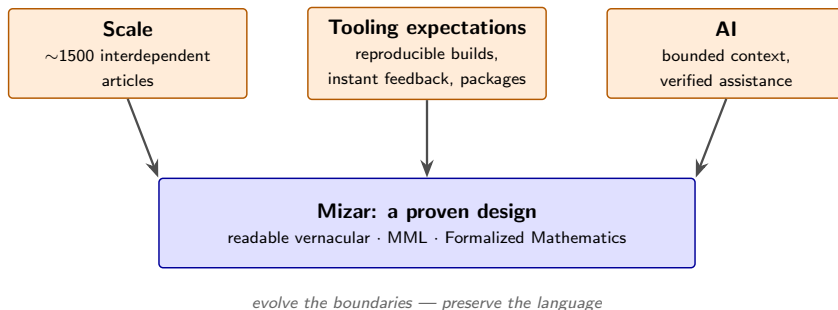
Message:

- Mizar's readability is not a nostalgic asset. It is exactly what makes safe AI assistance possible.

Presenter script: Read aloud:

- AI agents are already useful for search, explanation, and repair.
- They need bounded, structured, source-anchored context - not a dump of the whole library.
- Their output must never define proof truth; verification must stay independent of the strength of the assistant.
- Readable source is an advantage here: stable local text patterns are what AI edits and retrieval work best on.
- Mizar's readability is not a nostalgic asset. It is exactly what makes safe AI assistance possible.

1.5 - Three Pressures, One Design



Message:

- None of the three pressures argues that Mizar's design was wrong. Together they argue for evolving its boundaries.

**Message:**

- None of the three pressures argues that Mizar's design was wrong. Together they argue for evolving its boundaries.

Presenter script: Read aloud:

- None of the three pressures argues that Mizar's design was wrong. Together they argue for evolving its boundaries.

Key Phrase:

*Do not trade away readability to gain automation.
Use automation to protect and extend readability.*

Three pillars, one test:

Pillar	Test for every feature
Readability	does proof text still read as mathematics?
AI-readiness	can a tool see bounded, auditable context?
Scalability	do boundaries stay stable as the library grows?

*Do not to do away readability to go in automation.
Use automation to protect and extend readability.*

Three pillars, one test:

Pillar	Test for every feature
Readability	does your test still read as automation?
Abstraction	can a real one board it, and table contents?
Scalability	do boundaries stay walls as the library grows?

Presenter script: Say:

- The eight stories that follow each apply this rule to one concrete pain.

For the table, read the row labels first, then the design reason.

Legacy Mizar **exact MML excerpt**

`environ`

```
vocabularies XBOOLE_0, SUBSET_1, BINOP_1, ZFMISC_1, STRUCT_0, ARYTM_3,  
             FUNCT_1, FUNCT_5, SUPINF_2, ARYTM_1, RELAT_1, MESFUNC1, ALGSTR_0, CARD_1;  
notations TARSKI, XBOOLE_0, SUBSET_1, ZFMISC_1, BINOP_1, FUNCT_5, ORDINAL1,  
          CARD_1, STRUCT_0;  
constructors BINOP_1, STRUCT_0, ZFMISC_1, FUNCT_5;  
registrations ZFMISC_1, CARD_1, STRUCT_0;  
theorems STRUCT_0;
```

`begin :: Additive structures`

Part 2. Story 1: Dependencies You Can See

2.1 - The Pain

2.1 - The Pair

Legacy Mizar  MML KODIGO

REFERENCES

[illegible]

Presenter script: Say:

- Source: current MML algstr_0.miz, lines 15-25.
- Everyone in the room has edited one of these blocks by trial and error.
- The point is not that `environ` is bad; it successfully drove decades of library growth. The point is what it costs at today's scale.

Key Points:

- One symbol's origin is spread over several role lists; a reviewer cannot see which article contributes which notation, constructor, or cluster.
- The Accommodator resolves the environment, but the resolved surface is not reviewable source text.
- Tools cannot cache or invalidate at a finer granularity than the article.
- Moving a theorem between articles risks breaking unknown dependents.

Message:

- Implicit dependency surfaces are a fixed cost on every edit, every review, and every tool, and the cost grows with the library.

└ Part 2. Story 1: Dependencies You Can See

└ 2.2 - Why It Hurts

deep dive

2.2 - Why It Hurts

deep dive

Key Points:

- One symbol's origin is spread over several role lists; a reviewer cannot see which article contributes which notation, constructor, or cluster.
- The Accommodator resolves the environment, but the resolved surface is not reviewable source text.
- Tools cannot cache or invalidate at a finer granularity than the article.
- Moving a theorem between articles risks breaking unknown dependents.

Message:

- Implicit dependency surfaces are a fixed cost on every edit, every review, and every tool, and the cost grows with the library.

Presenter script: Read aloud:

- One symbol's origin is spread over several role lists; a reviewer cannot see which article contributes which notation, constructor, or cluster.
- The Accommodator resolves the environment, but the resolved surface is not reviewable source text.
- Tools cannot cache or invalidate at a finer granularity than the article.
- Moving a theorem between articles risks breaking unknown dependents.
- Implicit dependency surfaces are a fixed cost on every edit, every review, and every tool, and the cost grows with the library.

Mizar Evo **specification example**

```
import .function;  
import mml.algebra.structure.sorted;  
  
definition  
  let S be 1-sorted;  
  mode BinOpDef: BinOp of S is  
    Function of [: S.carrier, S.carrier :], S.carrier;  
end;
```

Rules that make this deterministic:

- all imports appear before the first item; no mid-file environment changes;
- imports seed the active lexicon, and every symbol, notation, registration, and theorem is traceable to exactly one import;
- stable fully-qualified names are derived from package and module paths.

Mizar Evo

└ Part 2. Story 1: Dependencies You Can See

└ 2.3 - The Evo Answer: Import Prelude

```

Mizar Evo specific example
import . functions;
import mm.assigner, secdecider, secdec;

definition
  let S be Sec-Code;
  mean SImpDef: SImp of S is
  Functions of S: SCarrier, SCarrier-1, SCarrier;
end;

```

Rules that make this deterministic:

- Imports appear before the first item; no mid-file environment changes;
- Imports seed the active lexicon, and every symbol, notation, registration, and theorem is traceable to exactly one import;
- stable fully-qualified names are derived from package and module paths.

Presenter script: Read aloud:

- all imports appear before the first item; no mid-file environment changes;
- imports seed the active lexicon, and every symbol, notation, registration, and theorem is traceable to exactly one import;
- stable fully-qualified names are derived from package and module paths.

After the example, say which design obligation the syntax makes visible.

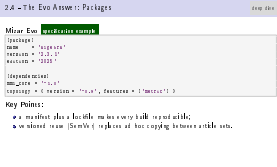
Mizar Evo **specification example**

```
[package]
name      = "algebra"
version   = "2.3.1"
edition   = "2025"

[dependencies]
mml_core  = "^1.0"
topology  = { version = "^0.9", features = ["metric"] }
```

Key Points:

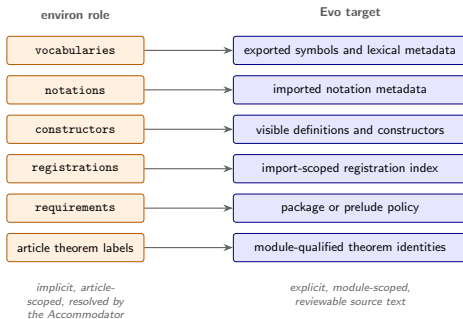
- a manifest plus a lockfile makes every build reproducible;
- versioned reuse (SemVer) replaces ad hoc copying between article sets.



Presenter script: Read aloud:

- a manifest plus a lockfile makes every build reproducible;
- versioned reuse (SemVer) replaces ad hoc copying between article sets.

After the example, say which design obligation the syntax makes visible.



Message:

- this is not a mechanical rename: current environments mix semantic, syntactic, and automation-facing roles, and migration reports must explain what each imported module actually contributes.

**Message:**

- this is not a mechanical rename: current environments mix semantic, syntactic, and automation-facing roles, and migration reports must explain what each imported module actually contributes.

Presenter script: Read aloud:

- this is not a mechanical rename: current environments mix semantic, syntactic, and automation-facing roles, and migration reports must explain what each imported module actually contributes.

Preserved:

- the mathematics and theorem identities are untouched;
- article-style authorship remains; a module is still a readable text;
- migration keeps origin metadata (story 8 returns to this).

Questions for Bialystok:

- Which `environ` roles must remain visually familiar during migration?
- Which current article dependencies are hardest to explain, and would make the best test cases for generated dependency reports?

└ Part 2. Story 1: Dependencies You Can See

└ 2.6 - What Is Preserved, What We Ask

Preserved:

- the mathematics and theorem identities are untouched;
- article-style authorship remains; a module is still a readable text;
- migration keeps origin metadata (story 8 returns to this).

Questions for Babyluk:

- Which environ roles must remain visually familiar during migration?
- Which current article dependencies are hardest to explain, and would make the best test cases for generated dependency reports?

Presenter script: Read aloud:

- the mathematics and theorem identities are untouched;
- article-style authorship remains; a module is still a readable text;
- migration keeps origin metadata (story 8 returns to this).
- Which environ roles must remain visually familiar during migration?
- Which current article dependencies are hardest to explain, and would make the best test cases for generated dependency reports?

Legacy Mizar **exact MML excerpt**

```
definition
  struct (1-sorted) addMagma (# carrier -> set, addF -> BinOp of the carrier
    #);
end;
```

Key Points:

- parent link, fields, and selector layout are one compact declaration;
- with multiple parents, the merge of inherited fields is implicit;
- renamed views (additive vs multiplicative) rest on naming conventions;
- stored data and canonical values (a zero, a unit) are not distinguished.

└ 3.1 - The Pain

Legacy Mix

```
(definition)
DEFAULT: [Source] addFlags: 18 uses: 50 SRC, BASE -> RUMBLE OF THE DEPTHS
M:
END;
```

Key Points:

- present *rich* fields, a set of which is layout as one or compact fields;
- split multiple *present* the merge of fields; fields is *infinite*;
- named items (additive or multiplicative) with no naming context;
- stored data as a canonical values (e.g. 20, 5) is not distinguished.

- Source: current MML algstr_0.miz, lines 37-40.
- Acknowledge that this compactness was a feature: structures stayed close to informal mathematical writing.

- parent link, fields, and selector layout are one compact declaration;
- with multiple parents, the merge of inherited fields is implicit;
- renamed views (additive vs multiplicative) rest on naming conventions;

Key Points:

- Diamond inheritance (one structure reachable through two parent paths) is resolved by convention and declaration order, not by checkable source.
- A migration tool cannot ask "is this selector intrinsic data, a canonical value with obligations, or an inherited view?" - the syntax does not say.
- Errors surface far from their cause, as type mismatches in later articles.

Message:

- At MML scale, structure inheritance is a graph maintenance problem, and the graph deserves explicit, checkable edges.

Key Points:

- Diamond inheritance (one structure reachable through two parent paths) is resolved by convention and declaration order, not by checkable source.
- A migration tool cannot ask "is this selector intrinsic data, a canonical value with obligations, or an inherited view?" - the syntax does not say.
- Errors surface far from their cause, as type mismatches in later articles.

Message:

- At MML scale, structure inheritance is a graph maintenance problem, and the graph deserves explicit, checkable edges.

Presenter script: Read aloud:

- Diamond inheritance (one structure reachable through two parent paths) is resolved by convention and declaration order, not by checkable source.
- A migration tool cannot ask "is this selector intrinsic data, a canonical value with obligations, or an inherited view?" - the syntax does not say.
- Errors surface far from their cause, as type mismatches in later articles.
- At MML scale, structure inheritance is a graph maintenance problem, and the graph deserves explicit, checkable edges.

Mizar Evo **specification example**

```
definition
  struct AddLoopStr where
    field carrier -> set;
    field add -> BinOp of carrier;
    property zero -> Element of carrier;
  end;
end;
```

Three distinct concepts:

Concept	Meaning	Consequence
field	intrinsic data supplied by the value	constructor shape, equality
property	uniquely determined canonical value	existence/uniqueness obligations
attribute	predicate-style refinement	cluster propagation, not layout

Mizar Evo

└ Part 3. Story 2: Structures Without Hidden Merges

└ 3.3 - The Evo Answer: Field, Property, Attribute

Mizar Evo **specific example**

```

def is in
  struct: Add supplier when
    field carrier -> set;
    field add -> Mimp of carrier;
    property set -> Element of carrier;
  end;
end;

```

Three distinct concepts:

Concept	Meaning	Consequence
field	Intrinsic data supplied by the value	commutative map, equality
property	value-dependent canonical value	extension/assignment obligations
attribute	predication with refinement	closure propagation, not known

Presenter script: After the example, say which design obligation the syntax makes visible.

For the table, read the row labels first, then the design reason.

Use this as the transition: The Evo Answer: Field, Property, Attribute. Point to the visible example, question, or diagram before moving on.

Mizar Evo **specification example**

```
definition
  struct AddMagma where
    field carrier -> set;
    field add -> BinOp of carrier;
  end;

  inherit AddMagma extends Magma where
    field carrier from carrier;
    field add from binop;      :: renamed
  end;
end;
```

Key Points:

- one inherit statement per parent; mapping and renaming are source text;
- narrowing and type conversions carry coherence proof obligations;
- the additive/multiplicative naming convention becomes a checked view.

└ Part 3. Story 2: Structures Without Hidden Merges

└ 3.4 - The Evo Answer: Explicit Inheritance

Mizar Evo **specific example**

```

definition
  structure addAgm name n
  field carrier -> set;
  field add -> natp of carrier;
  end;

inherit addAgm rec name Agm name n
  field carrier from carrier;
  field add from binop;  :: rmonoid
  end;
end;

```

Key Points:

- one inherit statement per parent; mapping and renaming are source text;
- narrowing and type conversions carry coherence proof obligations;
- the additive/multiplicative naming convention becomes a checked view.

Presenter script: Read aloud:

- one inherit statement per parent; mapping and renaming are source text;
- narrowing and type conversions carry coherence proof obligations;
- the additive/multiplicative naming convention becomes a checked view.

After the example, say which design obligation the syntax makes visible.

Mizar Evo **specification example**

```
struct DoubleLoopStr where
  field carrier -> set;
  field add -> BinOp of carrier;
  field mul -> BinOp of carrier;
  property zero -> Element of carrier;
  property one -> Element of carrier;
end;

inherit DoubleLoopStr extends AddLoopStr;
inherit DoubleLoopStr extends MulLoopStr;
```

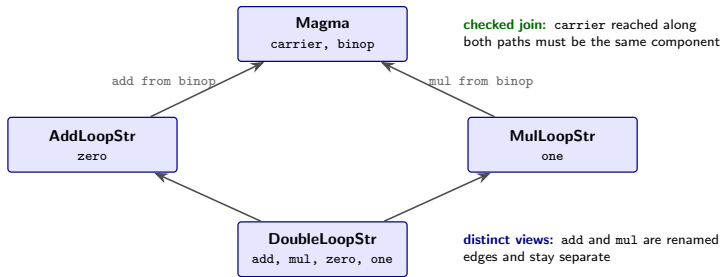
└ Part 3. Story 2: Structures Without Hidden Merges

└ 3.5 - The Evo Answer: Diamonds Become Checkable (1/2)

```
Mizar Evo specific example  
syntax: DoubleInteger <name>  
field current => int;  
field xax => sum <= current;  
field yax => sum <= current;  
property xax => sum <= current;  
name;  
inductive DoubleInteger <name> <double>  
inductive DoubleInteger <name> <double>
```

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: The Evo Answer: Diamonds Become Checkable (1/2). Point to the visible example, question, or diagram before moving on.

3.5 - The Evo Answer: Diamonds Become Checkable (2/2)



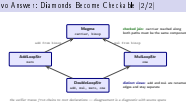
the verifier traces from chains to root declarations — disagreement is a diagnostic with source spans

Message:

- the verifier traces from chains to root declarations; both paths must introduce the same components (the shared carrier above);
- diamond inheritance becomes a diagnostic with source spans, not a silent merge decided by declaration order.

Part 3. Story 2: Structures Without Hidden Merges

3.5 - The Evo Answer: Diamonds Become Checkable (2/2)



Message:

- the verifier traces from chains to root declarations; both paths must introduce the same components (the shared carrier above);
- diamond inheritance becomes a diagnostic with source spans, not a silent merge decided by declaration order.

Presenter script: Read aloud:

- the verifier traces from chains to root declarations; both paths must introduce the same components (the shared carrier above);
- diamond inheritance becomes a diagnostic with source spans, not a silent merge decided by declaration order.

Preserved:

- structures remain Mizar structures: carriers, selectors, `Element of`;
- aggregate compactness is traded only where a hidden decision existed.

Questions for Bialystok:

- Is the `field / property / attribute` split readable in real algebraic articles, or does it over-annotate simple cases?
- Is `one-parent-per-inherit` acceptable for the inheritance-heavy parts of MML, such as the `ALGSTR` and topology hierarchies?
- Which MML structures would be the best diamond test cases?

└ Part 3. Story 2: Structures Without Hidden Merges

└ 3.6 - What Is Preserved, What We Ask

Preserved:

- structures remain Mizar structures: carriers, selectors, `MEMBER` etc;
- aggregate compactness is traded only where a hidden decision existed.

Questions for Babytok:

- Is the field / property / attribute split readable in real algebraic articles, or does it over-annotate simple cases?
- Is one-parent-per-inherit acceptable for the inheritance-heavy parts of MML, such as the ALGSTR and topology hierarchies?
- Which MML structures would be the best diamond test cases?

Presenter script: Read aloud:

- structures remain Mizar structures: carriers, selectors, Element of;
- aggregate compactness is traded only where a hidden decision existed.
- Is the field / property / attribute split readable in real algebraic articles, or does it over-annotate simple cases?
- Is one-parent-per-inherit acceptable for the inheritance-heavy parts of MML, such as the ALGSTR and topology hierarchies?
- Which MML structures would be the best diamond test cases?

Legacy Mizar **exact MML excerpt**

```
registration
  let M be addMagma;
  cluster right_add-cancelable left_add-cancelable -> add-cancelable for
Element
  of M;
  coherence;
end;
```

Mizar Evo

└ Part 4. Story 3: Automation You Can Audit

└ 4.1 - The Pain

4.1 - The Pain

Legacy Mizar `mml.html.miz`

```
begin formal use
  let M be addDiagram;
  cluster ex right_add <once table left_add <once table -> add-consistent for
  M use ex
  of M;
  coherence;
end;
```

Presenter script: Say:

- Source: current MML `algstr_0.miz`, lines 104-109.
- Registrations are one of Mizar's best ideas: adjectives propagate silently and proofs stay short. The pain is not the mechanism but its opacity.

Key Points:

- When a proof fails, "why does the checker not see that this is a Group?" has no local answer; the cause lives somewhere in the environment.
- Which registrations fired, in which order, is invisible; the automation is powerful, but its explanation does not scale with its power.
- For an AI assistant the situation is worse: it must guess the cluster state instead of reading it.

Message:

- Automation that cannot explain itself becomes a maintenance liability at library scale - even when it is sound.

4.2 - Why It Hurts

Key Points:

- When a proof fails, "why does the checker not see that this is a Group?" has no local answer; the cause lives somewhere in the environment.
- Which registrations fired, in which order, is invisible; the automation is powerful, but its explanation does not scale with its power.
- For an AI assistant the situation is worse: it must guess the cluster state instead of reading it.

Message:

- Automation that cannot explain itself becomes a maintenance liability at library scale - even when it is sound.

Presenter script: Read aloud:

- When a proof fails, "why does the checker not see that this is a Group?" has no local answer; the cause lives somewhere in the environment.
- Which registrations fired, in which order, is invisible; the automation is powerful, but its explanation does not scale with its power.
- For an AI assistant the situation is worse: it must guess the cluster state instead of reading it.
- Automation that cannot explain itself becomes a maintenance liability at library scale - even when it is sound.

Mizar Evo **specification example**

```
registration
  cluster EmptyImpliesFinite: empty -> finite for set;
  coherence proof ... end;

  cluster FiniteImpliesCountable: finite -> countable for set;
  coherence proof ... end;
end;
```

Key Points:

- every registration item carries a required label: citable in by, reported in diagnostics, part of the module interface;
- the verifier stores an import-filtered cluster resolution graph;
- @show_resolution and explanation artifacts answer "why (not)?" with the actual chain, e.g. empty -> finite -> countable.

└ Part 4. Story 3: Automation You Can Audit

└ 4.3 - The Evo Answer: Labeled, Traceable Registrations

Mizar Evo **specific example**

```

registration
  cluster EmptyImpl is Finite: empty -> finite for set;
  coherence proof ... end;

  cluster FiniteImpl is Countable: finite -> countable for set;
  coherence proof ... end;
end;

```

Key Points:

- every registration item carries a required label: citable in by, reported in diagnostics, part of the module interface;
- the verifier stores an import-filtered cluster resolution graph;
- @show_resolution and explanation artifacts answer "why (not)?" with the actual chain, e.g. empty -> finite -> countable.

Presenter script: Read aloud:

- every registration item carries a required label: citable in by, reported in diagnostics, part of the module interface;
- the verifier stores an import-filtered cluster resolution graph;
- @show_resolution and explanation artifacts answer "why (not)?" with the actual chain, e.g. empty -> finite -> countable.

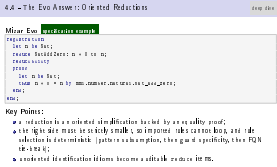
After the example, say which design obligation the syntax makes visible.

Mizar Evo **specification example**

```
registration
  let n be Nat;
  reduce NatAddZero:  $n + 0$  to  $n$ ;
  reducibility
  proof
    let n be Nat;
    thus  $n + 0 = n$  by mml.number.natural.Nat_add_zero;
  end;
end;
```

Key Points:

- a reduction is an oriented simplification backed by an equality proof;
- the right side must be strictly smaller, so imported rules cannot loop, and rule selection is deterministic (pattern subsumption, then guard specificity, then FQN tie-break);
- unoriented identification idioms become auditable reduce items.



Presenter script: Read aloud:

- a reduction is an oriented simplification backed by an equality proof;
- the right side must be strictly smaller, so imported rules cannot loop, and rule selection is deterministic (pattern subsumption, then guard specificity, then FQN tie-break);
- unoriented identification idioms become auditable reduce items.

After the example, say which design obligation the syntax makes visible.

Preserved:

- registrations and clusters remain first-class; proofs stay short;
- no new proof text is required at use sites - only traces are added.

Questions for Bialystok:

- Which cluster explanations would most reduce daily friction: failure explanations, firing traces, or difference reports between environments?
- Which MML article families stress registrations hardest and should become migration benchmarks for the cluster graph?

└ Part 4. Story 3: Automation You Can Audit

└ 4.5 - What Is Preserved, What We Ask

Preserved:

- registrations and clusters remain first-class; proofs stay short;
- no new proof text is required at use sites - only traces are added.

Questions for Babelok:

- Which cluster explanations would most reduce daily friction: failure explanations, firing traces, or difference reports between environments?
- Which MML article families stress registrations hardest and should become migration benchmarks for the cluster graph?

Presenter script: Read aloud:

- registrations and clusters remain first-class; proofs stay short;
- no new proof text is required at use sites - only traces are added.
- Which cluster explanations would most reduce daily friction: failure explanations, firing traces, or difference reports between environments?
- Which MML article families stress registrations hardest and should become migration benchmarks for the cluster graph?

Key Points:

- Users want stronger automation: bigger by steps, hammer-style search.
- But in a monolithic verifier, every gain in search power grows the code that must be trusted.
- External provers (ATPs) are strong exactly where Mizar's core is first-order - and they are the least auditable component of all.
- MizAR and the MPTP line of work already showed that ATP search is powerful on MML premises; the open question is trust, not power.

Key Phrase:

*How do we get modern proof search
without trusting the searcher?*

5.1 - The Pain

Key Points:

- Users want stronger automation: bigger by steps, hammer-style search.
- But in a monolithic verifier, every gain in search power grows the code that must be trusted.
- External provers (ATPs) are strong exactly where Mizar's core is first-order - and they are the least auditable component of all.
- Mizar as of the MPTP line of work already showed that ATP search is powerful on MML premises the user questions is trust, not power.

Key Phrase:

How do we get modern proofs such without trusting the searcher?

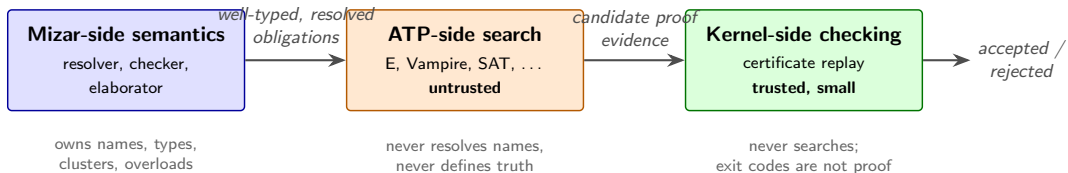
Presenter script: Say:

- Acknowledge MizAR, MPTP, and the hammer line of research explicitly: they demonstrated the search power on MML. Mizar Evo's contribution is the boundary that lets that power in without enlarging the trusted base.

Read aloud:

- Users want stronger automation: bigger by steps, hammer-style search.
- But in a monolithic verifier, every gain in search power grows the code that must be trusted.
- External provers (ATPs) are strong exactly where Mizar's core is first-order - and they are the least auditable component of all.

5.2 - The Evo Answer: A Reasoning Boundary



Key rules:

- ATPs never resolve names, infer types, expand clusters, or pick overloads;
- the kernel never searches for proofs;
- deterministic pre-ATP discharge needs replayable evidence too - nothing is accepted because an earlier phase said "done".

Part 5. Story 4: Powerful Search, Small Trust

5.2 - The Evo Answer: A Reasoning Boundary



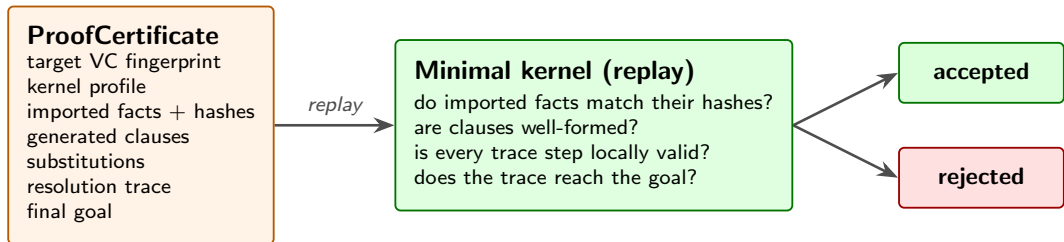
Key rules:

- ATPs never resolve names, infer types, expand clusters, or pick overloads;
- the kernel never searches for proofs;
- deterministic pre-ATP discharge needs replayable evidence too - nothing is accepted because an earlier phase said "done".

Presenter script: Read aloud:

- ATPs never resolve names, infer types, expand clusters, or pick overloads;
- the kernel never searches for proofs;
- deterministic pre-ATP discharge needs replayable evidence too - nothing is accepted because an earlier phase said "done".

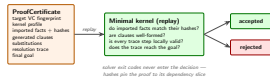
5.3 - Certificates, Not Success Bits (1/2)



*solver exit codes never enter the decision —
hashes pin the proof to its dependency slice*

Part 5. Story 4: Powerful Search, Small Trust

5.3 - Certificates, Not Success Bits (1/2)



Presenter script: Use this as the transition: Certificates, Not Success Bits (1/2). Point to the visible example, question, or diagram before moving on.

Key Points:

- accepted evidence is normalized into replayable certificate data;
- the kernel checks imported facts, substitutions, clause well-formedness, and the resolution/SAT trace - it does not trust a solver's exit code, so unsoundness in search cannot become unsoundness in accepted results;
- a proof accepted under one dependency slice cannot silently migrate to another: the hashes pin it down.

└ Part 5. Story 4: Powerful Search, Small Trust

└ 5.3 - Certificates, Not Success Bits (2/2)

Key Points:

- accepted evidence is normalized into replayable certificate data;
- the kernel checks imported facts, substitutions, clause well-formedness, and the resolution/SAT trace - it does not trust a solver's exit code, so unsoundness in search cannot become unsoundness in accepted results;
- a proof accepted under one dependency slice cannot silently migrate to another: the hashes pin it down.

Presenter script: Read aloud:

- accepted evidence is normalized into replayable certificate data;
- the kernel checks imported facts, substitutions, clause well-formedness, and the resolution/SAT trace - it does not trust a solver's exit code, so unsoundness in search cannot become unsoundness in accepted results;
- a proof accepted under one dependency slice cannot silently migrate to another: the hashes pin it down.

Legacy Mizar **exact MML excerpt**

```
theorem
  for F being non degenerated ZeroOneStr holds 1.F in NonZero F
proof
  let F be non degenerated ZeroOneStr;
  not 1.F in {0.F} by TARSKI:def 1;
  hence thesis by XBOOLE_0:def 5;
end;
```

Message:

- Citation repair - proposing a missing or sharper by reference - is the canonical safe AI edit: source-local, meaning-preserving, and checked by the verifier like any human edit.

└ Part 5. Story 4: Powerful Search, Small Trust

└ 5.4 - The Same Boundary Tames AI

Legacy Mizar **mml, mml, mml**

```

theorem
  for P being non degenerated LoxtonStr holds 1.P in the same P
proof
  let P be non degenerated LoxtonStr;
  not 1.P in (1.P) by TMSK:1def 1;
  hence 1.P is by XMSK_1:1def 1;
end;

```

Message:

- Citation repair - proposing a missing or sharper by reference - is the canonical safe AI edit: source-local, meaning-preserving, and checked by the verifier like any human edit.

Presenter script: Say:

- Source: current MML struct_0.miz, lines 637-643.
- The agent proposes; the verifier and kernel decide. The assistant's strength never enters the trusted base.

Read aloud:

- Citation repair - proposing a missing or sharper by reference - is the canonical safe AI edit: source-local, meaning-preserving, and checked by the verifier like any human edit.

Class	Examples	Policy
Green	add citation, insert qua, info annotation	auto-proposable, still verified
Yellow	add import, local lemma, registration	proposed with human review
Red	weaken theorem, change definition, add axiom	forbidden to ordinary agents

Forbidden repair **sketch**

theorem

```
for x be Nat holds x + 0 = x or x = 0;
```

Message:

- Weakening a statement to make its proof easier is a Red edit; at most an agent may flag it for explicit human unsafe-edit review.

└ Part 5. Story 4: Powerful Search, Small Trust

└ 5.5 - Edit Classes: Green, Yellow, Red

deep dive

5.5 - Edit Classes: Green, Yellow, Red deep dive

Class	Example	Policy
Green	add division, insert ops, into and oracles	auto-approvals, still verified
Yellow	add imports, local defs, a, register to	proposed with human review
Red	modify theorem, change definition, add axiom	forbidden to ordinary agents

Forbidden repair new

```

theorem
  for x be Nat holds x + 0 = x and x + 0 = 0;
  
```

Message:

- ⚠ Weakening a statement to make its proof easier is a Red edit; at most an agent may flag it for explicit human unsafe-edit review.

Presenter script: Read aloud:

- Weakening a statement to make its proof easier is a Red edit; at most an agent may flag it for explicit human unsafe-edit review.

After the example, say which design obligation the syntax makes visible.

For the table, read the row labels first, then the design reason.

Preserved:

- the de Bruijn discipline: a small checker judges everything;
- proof text stays declarative and readable - automation maintains the argument, it does not replace it.

Questions for Bialystok:

- Is the SAT/resolution certificate story convincing for Mizar-style obligations, including clusters and definitional expansions?
- Which evidence format would the team be most willing to audit?

Preserved:

- the de Bruijn discipline: a small checker judges everything
- proof text stays declarative and readable - automation maintains the argument, it does not replace it.

Questions for Balabot:

- Is the SAT/resolution certificate story convincing for Mizar-style obligations, including clusters and definitional expansions?
- Which evidence format would the team be most willing to audit?

Presenter script: Read aloud:

- the de Bruijn discipline: a small checker judges everything;
- proof text stays declarative and readable - automation maintains the argument, it does not replace it.
- Is the SAT/resolution certificate story convincing for Mizar-style obligations, including clusters and definitional expansions?
- Which evidence format would the team be most willing to audit?

Key Points:

- Verifying the whole MML is a batch operation measured in hours.
- The reuse boundary is the accepted article: a small change re-verifies more than it should.
- Memory follows the article environment, not the actually used interface.
- None of this is a defect of the current design - it is what article-level granularity implies once the library is large.

Key Points:

- Verifying the whole MML is a batch operation measured in hours.
- The reuse boundary is the accepted article: a small change re-verifies more than it should.
- Memory follows the article environment, not the actually used interface.
- None of this is a defect of the current design - it is what article-level granularity implies once the library is large.

Presenter script: Read aloud:

- Verifying the whole MML is a batch operation measured in hours.
- The reuse boundary is the accepted article: a small change re-verifies more than it should.
- Memory follows the article environment, not the actually used interface.
- None of this is a defect of the current design - it is what article-level granularity implies once the library is large.

Key Points:

- hash the source, the dependency interfaces, the generated obligations, and the proof witnesses;
- reuse a result only when fingerprints and verifier policy match;
- a proof-body-only change does not rebuild importers, because public statements and statuses are unchanged;
- independent modules, obligations, ATP runs, and kernel checks run in parallel; results are published in canonical order.

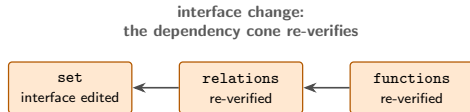
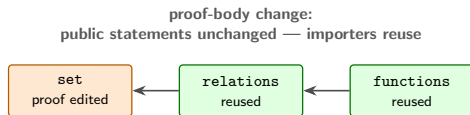
Key Points:

- hash the source, the dependency interfaces, the generated obligations, and the proof witnesses;
- reuse a result only when fingerprints and verifier policy match;
- a proof-body-only change does not rebuild importers, because public statements and statuses are unchanged;
- independent modules, obligations, ATP runs, and kernel checks can be parallel results are published in canonical order.

Presenter script: Read aloud:

- hash the source, the dependency interfaces, the generated obligations, and the proof witnesses;
- reuse a result only when fingerprints and verifier policy match;
- a proof-body-only change does not rebuild importers, because public statements and statuses are unchanged;
- independent modules, obligations, ATP runs, and kernel checks run in parallel; results are published in canonical order.

6.2 - The Evo Answer: Fingerprints And Incrementality (2/2)



reuse requires matching fingerprints: source, dependency interfaces, obligations, proof witnesses — and a clean build must reproduce every acceptance

Rule:

*Cache reuse is never proof authority.
A clean build must always be able to reproduce every acceptance.*

└ Part 6. Story 5: Verification That Scales

└ 6.2 - The Evo Answer: Fingerprints And Incrementality (2/2)



Rule:

Cache reuse is never proof authority.
A class build must always be able to reproduce every acceptance.

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: The Evo Answer: Fingerprints And Incrementality (2/2). Point to the visible example, question, or diagram before moving on.

resident memory should scale with:

- active source
- imported public interfaces
- import-filtered indexes
- active module obligations

not with:

- imported proof bodies
- private lemmas outside the interface
- registration data outside the import closure

Message:

- Interfaces are loaded; proof bodies are not. This is what makes whole-MML editing sessions feasible on ordinary hardware.

```

PROVERID: MIZARID PROVERID: MIZARID
ACCUIN: MIZARID
IMPORTED: PUBLIC: IMPORTED
IMPORTED: PUBLIC: IMPORTED
IMPORTED: MIZARID: MIZARID
IMPORTED: MIZARID: MIZARID
END: MIZARID:
IMPORTED: PROVERID: MIZARID
PROVERID: MIZARID: MIZARID
PROVERID: MIZARID: MIZARID

```

Message:

• This is what makes whole-MML editing sessions feasible on ordinary hardware.

Presenter script: Read aloud:

- Interfaces are loaded; proof bodies are not. This is what makes whole-MML editing sessions feasible on ordinary hardware.

After the example, say which design obligation the syntax makes visible.

Preserved:

- clean-build semantics: caching and parallelism change speed, never truth;
- article-style review: what a human reads is still complete source text.

Questions for Bialystok:

- What clean-build equivalence tests would make incremental verification trustworthy to the team that maintains MML today?
- Which current maintenance operations (revisions, renamings) should we benchmark for incremental cost?

Preserved:

- clean-build semantics: caching and parallelism change speed, never truth;
- article-style review: what a human reads is still complete source text.

Questions for Babyluk:

- What clean-build equivalence tests would make incremental verification trustworthy to the team that maintains MML today?
- Which current maintenance operations (revisions, renamings) should we benchmark for incremental cost?

Presenter script: Read aloud:

- clean-build semantics: caching and parallelism change speed, never truth;
- article-style review: what a human reads is still complete source text.
- What clean-build equivalence tests would make incremental verification trustworthy to the team that maintains MML today?
- Which current maintenance operations (revisions, renamings) should we benchmark for incremental cost?

Legacy Mizar **exact MML excerpt**

```
scheme
  NatInd { P[Nat] } : for k being Nat holds P[k]
provided
A1: P[0] and
A2: for k be Nat st P[k] holds P[k + 1]
```

Key Points:

- schemes carry second-order patterns (induction, separation, replacement) and they work - but they are a separate mechanism with separate rules;
- schemes can parameterize theorems, but not structures, modes, or functors.

└ Part 7. Story 6: Templates For Generic Mathematics

└ 7.1 - The Pain, Part One: Schemes Are Fenced Off

```
schemes
natInd < P1Nat1 > : for k being Nat holds P1k
prv v1d1 d
Ab: P101: nat1
Ad: for k be Nat st P1k holds P1k + 1
```

Key Points:

- schemes carry second-order patterns (induction, separation, replacement) and they work - but they are a separate mechanism with separate rules;
- schemes can parameterize theorems, but not structures, modes, or functors.

Presenter script: Say:

- Source: current MML nat_1.miz, near line 90 (checked July 2, 2026).

Read aloud:

- schemes carry second-order patterns (induction, separation, replacement) and they work - but they are a separate mechanism with separate rules;
- schemes can parameterize theorems, but not structures, modes, or functors.

Key Points:

- `addMagma` and `multMagma` are the same mathematics twice, related only by naming convention (story 2 met this already);
- polynomial rings, vector spaces, and matrix theories are re-spelled per carrier because there is no parameterized construction;
- a theorem proved for one commutative operation is re-proved for `+` and `*` separately.

Message:

- The library pays for the missing generics mechanism in duplicated articles, and every duplicate is a maintenance obligation.

└ Part 7. Story 6: Templates For Generic Mathematics

└ 7.2 - The Pain, Part Two: Copy-Paste Algebra

7.2 - The Pain, Part Two: Copy-Paste Algebra
Key Points: • <code>addMagma</code> and <code>multMagma</code> are the same mathematics twice, related only by naming convention (story 2 met this already); • polynomial rings, vector spaces, and matrix theories are re-spelled per carrier because there is no parameterized construction; • a theorem proved for one commutative operation is re-proved for $+$ and $*$ separately.
Message: • The library pays for the missing generics mechanism in duplicated articles, and every duplicate is a maintenance obligation.

Presenter script: Read aloud:

- `addMagma` and `multMagma` are the same mathematics twice, related only by naming convention (story 2 met this already);
- polynomial rings, vector spaces, and matrix theories are re-spelled per carrier because there is no parameterized construction;
- a theorem proved for one commutative operation is re-proved for $+$ and $*$ separately.
- The library pays for the missing generics mechanism in duplicated articles, and every duplicate is a maintenance obligation.

Mizar Evo **specification example**

```
definition
  let T be type;
  struct MagmaStr[T] where
    field carrier -> T;
    field binop -> BinOp of T;
  end;
end;
```

Key Points:

- a template is an ordinary definition block whose leading `let` binds parameters: types, values, predicates, or functors;
- one mechanism covers structures, modes, functors, predicates, theorems, registrations, and algorithms;
- readable shorthands survive: `Module over R` is an automatic synonym for `Module[R]`, `Subset of X` for `Subset[X]`.

```

def is it is n
  let T be type;
  assume Magma (T) where
    f is id carrier -> T;
    f is id binary -> Binop of T;
  end;
end;

```

Key Points:

- a template is an ordinary definition block whose leading let binds parameters: types, values, predicates, or functors;
- one mechanism covers structures, modes, functors, predicates, theorems, registrations, and algorithms;
- readable shorthands survive: Module over R is an automatic synonym for Module[R], Subset of X for Subset[X].

Presenter script: Read aloud:

- a template is an ordinary definition block whose leading let binds parameters: types, values, predicates, or functors;
- one mechanism covers structures, modes, functors, predicates, theorems, registrations, and algorithms;
- readable shorthands survive: Module over R is an automatic synonym for Module[R], Subset of X for Subset[X].

After the example, say which design obligation the syntax makes visible.

Mizar Evo **specification example**

```
definition
  let T be type extends commutative Magma;
  theorem PermProduct[T]:
    for s being FinSequence of T,
      p being Permutation of dom s
    holds Product[T](s) = Product[T](s * p)
  proof ... end;
end;
```

Message:

- type extends commutative Magma states what the parameter must provide before any proof search begins;
- the theorem is proved once, for every commutative operation.

Mizar Evo **newBroune example**

```

defined on
  let T be type, assume a commutative magma;
  theorem ParamProduct111:
    for a being FinSequence of T,
      p being Permutation of dom a
    holds Product111(a) = Product111(a * p)
  prove ... end;
end;

```

Message:

- type extends commutative Magma states what the parameter must provide before any proof search begins;
- the theorem is proved once, for every commutative operation.

Presenter script: Read aloud:

- type extends commutative Magma states what the parameter must provide before any proof search begins;
- the theorem is proved once, for every commutative operation.

After the example, say which design obligation the syntax makes visible.

Instantiation **specification example**

```
PermProduct[AddMagma]           :: additive form
PermProduct[commutative MulMagma] :: multiplicative form

let R be commutative Ring;
PermProduct[R qua AddMagma]      :: R's additive view
PermProduct[R qua MulMagma]     :: R's multiplicative view
```

Message:

- qua selects the intended view when a ring reaches Magma along two paths - and notation follows the view: the generic * displays as + under the additive instantiation.

└ Part 7. Story 6: Templates For Generic Mathematics

└ 7.5 - One Proof, Many Instantiations

deep dive

**Presenter script: Read aloud:**

- qua selects the intended view when a ring reaches Magma along two paths - and notation follows the view: the generic * displays as + under the additive instantiation.

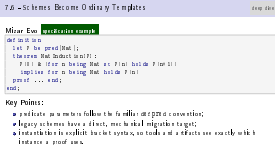
After the example, say which design obligation the syntax makes visible.

Mizar Evo specification example

```
definition
  let P be pred(Nat);
  theorem NatInduction[P]:
    P(0) & (for n being Nat st P(n) holds P(n+1))
    implies for n being Nat holds P(n)
  proof ... end;
end;
```

Key Points:

- predicate parameters follow the familiar defpred convention;
- legacy schemes have a direct, mechanical migration target;
- instantiation is explicit bracket syntax, so tools and artifacts see exactly which instance a proof uses.



Presenter script: Read aloud:

- predicate parameters follow the familiar defpred convention;
- legacy schemes have a direct, mechanical migration target;
- instantiation is explicit bracket syntax, so tools and artifacts see exactly which instance a proof uses.

After the example, say which design obligation the syntax makes visible.

Preserved:

- scheme-style reasoning survives unchanged in power;
- of / over phrasing keeps mathematical prose readable;
- first-order discipline: templates are checked instantiation, not a new logic.

Questions for Bialystok:

- Which MML schemes should be the first migration targets?
- Are brackets acceptable as the canonical identity form, with of/over as display forms?
- The specification settles inference conservatively: a parameter is inferred only when the declared argument types determine it uniquely, and qua views are never inferred. Is that conservatism right for real MML idioms, or does it demand too many explicit [T]?

└ Part 7. Story 6: Templates For Generic Mathematics

└ 7.7 - What Is Preserved, What We Ask

Preserved:

- scheme-style reasoning survives unchanged in power;
- *of / over* phrasing keeps mathematical prose readable;
- first-order discipline: templates are checked instantiation, not a new logic.

Questions for Babelok:

- Which MML schemes should be the first migration targets?
- Are brackets acceptable as the canonical identity form, with *of/over* as display forms?
- The specification settles informally, a parameter is inferred only when its declared signature type admits it uniquely, a *let* quantifier is inferred. Is that common enough for real MML idioms, or does it demand too many explicit [?]

Presenter script: Read aloud:

- scheme-style reasoning survives unchanged in power;
- *of / over* phrasing keeps mathematical prose readable;
- first-order discipline: templates are checked instantiation, not a new logic.
- Which MML schemes should be the first migration targets?
- Are brackets acceptable as the canonical identity form, with *of/over* as display forms?

Key Points:

- Current Mizar can define Gcd and prove its theory - but cannot compute `gcd(48, 18)`; every numeric fact needs a hand-written proof chain.
- There is no checked connection between MML mathematics and executable code; verified-algorithm work must leave the system entirely.
- This is a boundary of the design, not a defect: Mizar chose to be a proof language. The question is whether that boundary still serves us.

Key Phrase:

*The library describes computation.
It cannot perform or export it.*

└ Part 8. Story 7: Verified Computation With Algorithms

└ 8.1 - The Pain

8.1 - The Pain

Key Points:

- Current Mizar can define Gcd and prove its theory - but cannot compute $\text{gcd}(48, 18)$; every numeric fact needs a hand-written proof chain.
- There is no checked connection between MML mathematics and executable code; verified-algorithm work must leave the system entirely.
- This is a boundary of the design, not a defect: Mizar chose to be a proof language. The question is whether that boundary still serves us.

Key Phrase:

The library describes computation, it cannot perform or export it.

Presenter script: Read aloud:

- Current Mizar can define Gcd and prove its theory - but cannot compute $\text{gcd}(48, 18)$; every numeric fact needs a hand-written proof chain.
- There is no checked connection between MML mathematics and executable code; verified-algorithm work must leave the system entirely.
- This is a boundary of the design, not a defect: Mizar chose to be a proof language. The question is whether that boundary still serves us.

After the example, say which design obligation the syntax makes visible.

Mizar Evo **specification example** (condensed from spec section 20.12)

```
definition
  let a, b be Nat;
  terminating algorithm euclid_gcd(a, b) -> Nat
    requires a >= 1 & b >= 1
    ensures result = Gcd(a, b)
  do
    var x := a; var y := b;
    while y <> 0 do
      invariant x >= 1 & y >= 0 & Gcd(a, b) = Gcd(x, y);
      decreasing y;
      const r := x mod y; x := y; y := r;
    end;
    return x;
  end;
end;
```

Message:

- ensures and the invariant cite the mathematical Gcd: no circularity.

└ Part 8. Story 7: Verified Computation With Algorithms

└ 8.2 - The Evo Answer: Algorithms With Contracts

Mizar Evo [specifics example](#) (condensed from [spac machine 28.03](#))

```

definition
  let a, b be Nat;
  terminating algorithm gcd_algo, bl => Nat
  requires a > 0 & b > 0
  ensures result = gcd(a, b)
do
  let x := a; let y := b;
  while y <= 0 do
    decreasing x > 0 & a > y > 0 & a <= a, bl = gcd(a, y);
    decreasing y;
    let x := x mod y; a := x; y := y;
  end;
  result := x;
end;
end;

```

Message:
 • ensures and the invariant cite the mathematical Gcd: no circularity

Presenter script: Say:

- The contract speaks the mathematical language: the algorithm computes, while the library functor Gcd specifies. Termination comes from the decreasing measure on y.

Read aloud:

- ensures and the invariant cite the mathematical Gcd: no circularity.

Mizar Evo **specification example**

```
theorem EuclidGcd12_8:  euclid_gcd(12, 8)  = 4  by computation;  
theorem EuclidGcd100_75: euclid_gcd(100, 75) = 25 by computation;  
  
theorem Fact10: factorial(10) = 3628800  
proof  
  thus thesis by computation(steps: 100000);  
end;
```

Key Points:

- the Mizar Virtual Machine (MVM) evaluates ground goals during verification, under explicit step, time, and depth budgets;
- only ground equalities and ground predicates qualify; everything else still requires a classical proof;
- imported algorithms are opaque: downstream proofs may use only their ensures contract, never their body.

└ Part 8. Story 7: Verified Computation With Algorithms

└ 8.3 - The Evo Answer: Proof By Computation

Mizar Evo **specific example**

```

theorem RealIsInt0_0: succ(succ(succ(0), 0)) = 4 by computation;
theorem RealIsInt0_70: succ(succ(succ(0), 70)) = 72 by computation;

theorem Fact100: factoring(100) = 2 0200 00
proof
  show this is by computation in scope: 100 00 0;
end;

```

Key Points:

- the Mizar Virtual Machine (MVM) evaluates ground goals using verification, under explicit step, time, and depth budgets;
- only ground equalities and ground predicates qualify; everything else still requires a classical proof;
- imported algorithms are opaque: downstream proofs may use only their ensures contract, never their body.

Presenter script: Read aloud:

- the Mizar Virtual Machine (MVM) evaluates ground goals during verification, under explicit step, time, and depth budgets;
- only ground equalities and ground predicates qualify; everything else still requires a classical proof;
- imported algorithms are opaque: downstream proofs may use only their ensures contract, never their body.

After the example, say which design obligation the syntax makes visible.

Mizar Evo **specification example**

```
definition
  let n be Nat;
  terminating algorithm factorial(n) -> Nat
    decreasing n
  do
    if n = 0 do return 1; end;
    return n * factorial(n - 1);
  end;
end;
```

Key Points:

- ordinary func definitions are definitional extensions: never recursive;
- once its termination obligations are discharged, a terminating algorithm is promoted to a genuine functor, usable in any proof;
- this is the only door through which recursion enters the mathematical layer, and it is a proof-shaped door.

Mizar Evo **specification example**

```

definitions
  let m be Nat;
  terminating algorithm Success(m) -> Nat
  decreasing on
  do
    let n = m do return 1; end;
  return m + Success(n) - 1;
end;
end;

```

Key Points:

- ordinary func definitions are definitional extensions: never recursive;
- once its termination obligation is discharged, a terminating algorithm is promoted to a genuine functor, usable in any proof;
- this is the only door through which recursion enters the mathematical layer, and it is a proof-shaped door.

Presenter script: Read aloud:

- ordinary func definitions are definitional extensions: never recursive;
- once its termination obligations are discharged, a terminating algorithm is promoted to a genuine functor, usable in any proof;
- this is the only door through which recursion enters the mathematical layer, and it is a proof-shaped door.

After the example, say which design obligation the syntax makes visible.

Key Points:

- contracts, invariants, and termination measures generate verification conditions that pass through the same ATP-plus-kernel boundary as ordinary theorems (story 4);
- by computation is MVM replay under budgets, not solver trust;
- code extraction (to runtime targets) is strictly downstream of verified artifacts and can never feed back into acceptance.

Message:

- Algorithms widen what the library can express; they do not touch what it means for a theorem to be accepted.

└ Part 8. Story 7: Verified Computation With Algorithms

└ 8.5 - Computation Never Redefines Truth

deep dive

Key Points:

- contracts, invariants, and termination measures generate verification conditions that pass through the same ATP-plus-kernel boundary as ordinary theorems (story 4);
- by computation is MVM replay under budgets, not solver trust;
- code extraction (to runtime targets) is strictly downstream of verified artifacts and can never feed back into acceptance.

Message:

- Algorithms widen what the library can express; they do not touch what it means for a theorem to be accepted.

Presenter script: Read aloud:

- contracts, invariants, and termination measures generate verification conditions that pass through the same ATP-plus-kernel boundary as ordinary theorems (story 4);
- by computation is MVM replay under budgets, not solver trust;
- code extraction (to runtime targets) is strictly downstream of verified artifacts and can never feed back into acceptance.
- Algorithms widen what the library can express; they do not touch what it means for a theorem to be accepted.

Preserved:

- the theorem language is unchanged; algorithms live in `definition` blocks and interact with proofs only through contracts and `verified promotion`;
- first-order set-theoretic foundations stay intact.

Questions for Bialystok:

- Which computational examples would demonstrate value to mathematicians without shifting the culture toward programming?
- Are there MML areas (number theory, combinatorics, finite structures) where by `computation` would immediately shorten real proofs?
- Which extraction targets matter first, if any?

└ Part 8. Story 7: Verified Computation With Algorithms

└ 8.6 - What Is Preserved, What We Ask

Preserved:

- the theorem language is unchanged; algorithms live in definition blocks and interact with proofs only through contracts and verified promotion;
- first-order set-theoretic foundations stay intact.

Questions for Balabek:

- Which computational examples would demonstrate value to mathematicians without shifting the culture toward programming?
- Are there MML areas (number theory, combinatorics, finite structures) where by computation would immediately shorten real proofs?
- Which extraction targets matter first, if any?

Presenter script: Read aloud:

- the theorem language is unchanged; algorithms live in definition blocks and interact with proofs only through contracts and verified promotion;
- first-order set-theoretic foundations stay intact.
- Which computational examples would demonstrate value to mathematicians without shifting the culture toward programming?
- Are there MML areas (number theory, combinatorics, finite structures) where by computation would immediately shorten real proofs?
- Which extraction targets matter first, if any?

Key Points:

- Formalized Mathematics gives Mizar something rare: a scholarly, citable publication layer over a formal library.
- But article identity and library organization are tightly coupled: refactoring the library strains published article structure, and package reuse has no journal-facing identity at all.
- Exposition wants narrative order; reuse wants dependency order. One structure cannot optimize both.

Mizar Evo

└ Part 9. Story 8: A Library You Can Cite

└ 9.1 - The Pain

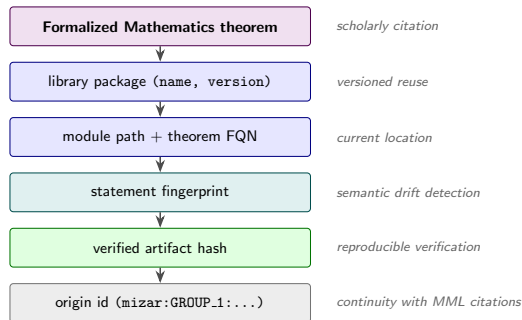
Key Points:

- Formalized Mathematics gives Mizar something rare: a scholarly, citable publication layer over a formal library.
- But article identity and library organization are tightly coupled: refactoring the library strains published article structure, and package reuse has no journal-facing identity at all.
- Exposition wants narrative order; reuse wants dependency order. One structure cannot optimize both.

Presenter script: Read aloud:

- Formalized Mathematics gives Mizar something rare: a scholarly, citable publication layer over a formal library.
- But article identity and library organization are tightly coupled: refactoring the library strains published article structure, and package reuse has no journal-facing identity at all.
- Exposition wants narrative order; reuse wants dependency order. One structure cannot optimize both.

9.2 - The Evo Answer: Linked, Not Merged



Message:

- each layer answers a different question: scholarly citation, current location, semantic drift detection, reproducible verification, and historical continuity with MML and past Formalized Mathematics volumes.

**Message:**

- each layer answers a different question: scholarly citation, current location, semantic drift detection, reproducible verification, and historical continuity with MML and past Formalized Mathematics volumes.

Presenter script: Read aloud:

- each layer answers a different question: scholarly citation, current location, semantic drift detection, reproducible verification, and historical continuity with MML and past Formalized Mathematics volumes.

Audience	Gain
readers	prose stays primary; formal source is one click away
maintainers	refactoring no longer rewrites published exposition
authors	articles cite stable identities, not file layouts
AI tools	prose for retrieval, fingerprints for exact context

└ Part 9. Story 8: A Library You Can Cite

└ 9.3 - Who Gains What

deep dive

9.3 - Who Gains What	
Audience	Gain
readers	promote new primary/secondary sources in one click
manipulators	reducing no longer running past books
authors	avoid the work in fiction, not the books
Alfred	promote the individual, to engage in the next content

Presenter script: For the table, read the row labels first, then the design reason. Use this as the transition: Who Gains What. Point to the visible example, question, or diagram before moving on.

Preserved:

- Formalized Mathematics remains a real journal with review and exposition;
- origin metadata keeps continuity with every existing MML citation.

Questions for Bialystok:

- Which identity should be primary in user-facing citations: article label, library FQN, or origin id?
- How should existing Formalized Mathematics articles link to migrated modules - retroactively, on revision, or not at all?

└ Part 9. Story 8: A Library You Can Cite

└ 9.4 - What Is Preserved, What We Ask

Preserved:

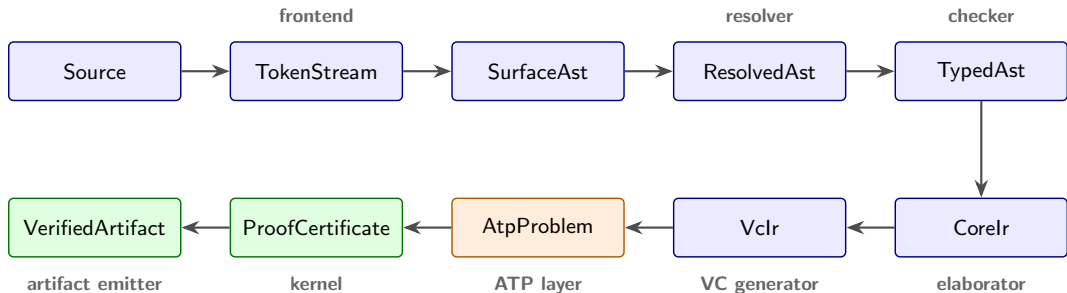
- Formalized Mathematics remains a real journal with review and exposition;
- origin metadata keeps continuity with every existing MML citation.

Questions for Babelotok:

- Which identity should be primary in user-facing citations: article label, library FQN, or origin id?
- How should existing Formalized Mathematics articles link to migrated modules - retroactively or not at all?

Presenter script: Read aloud:

- Formalized Mathematics remains a real journal with review and exposition;
- origin metadata keeps continuity with every existing MML citation.
- Which identity should be primary in user-facing citations: article label, library FQN, or origin id?
- How should existing Formalized Mathematics articles link to migrated modules - retroactively, on revision, or not at all?



Message:

- every boundary states who owns a fact, which artifact records it, and what must be recomputed when it changes - that is the entire point.

**Message:**

- every boundary states who owns a fact, which artifact records it, and what must be recomputed when it changes - that is the entire point.

Presenter script: Read aloud:

- every boundary states who owns a fact, which artifact records it, and what must be recomputed when it changes - that is the entire point.

Layer	Responsibility
frontend	lexing, parsing, recovery
resolver	imports, names, labels, namespaces
checker	soft types, clusters, registrations, overloads
elaborator	core logical representation
VC generator	proof and algorithm obligations
ATP layer plus kernel	untrusted search, then acceptance by checking
artifact emitter	stable outputs for tools and dependents

10.2 - Responsibility Split	
Layer	Design rationale
external	binding, parsing, execution
resolver	in parts, nam ex, binds, nam space
checker	mlt: type m, closure, registration, overloads
dataflow	core logical representation
VC generation	proof and algorithm equivalence
ATP layer placeholder	external match, then acceptance by checking
interface layer	enable outputs for each and dependent

Presenter script: For the table, read the row labels first, then the design reason. Use this as the transition: Responsibility Split. Point to the visible example, question, or diagram before moving on.

Story	Pipeline home
dependencies	resolver, package manager
structures and automation	checker (inheritance and cluster graphs, traces)
search vs trust	ATP layer, kernel, certificates
scale	artifacts, fingerprints, scheduler
templates	elaborator (checked instantiation)
algorithms	VC generator, MVM
publication	artifact emitter, doc generation

Message:

- the stories are not eight separate projects; they are one pipeline seen from eight user-visible pains.

└ Part 10. Architecture In One Picture

└ 10.3 - Where The Eight Stories Live

Story	Pipeline name
dependencies	resolver, package manager
translation and annotation	checker (dependencies and data graphs, traced)
module reuse	OTF 2 flow, kernel, configuration
work	compiler, 5 engine, scheduler
templates	dataflow (checked in translation)
optimization	VC generator, MM
publication	artifact resolution, data generation

Message:

- the stories are not eight separate projects; they are one pipeline seen from eight user-visible pains.

Presenter script: Read aloud:

- the stories are not eight separate projects; they are one pipeline seen from eight user-visible pains.

For the table, read the row labels first, then the design reason.

Key Phrase:

*Reject what must not pass
before
Accept everything that should pass*

Key Points:

- soundness bugs outrank parser gaps; kernel-adjacent tests emphasize malformed and failing evidence first;
- accepted-language coverage grows behind that shield.

10.4 - Testing The Trust Boundary

Key Phrase:

*Reject what must not pass
Before
Accept everything that should pass*

Key Points:

- soundness bugs outrank parser gaps; kernel-adjacent tests emphasize malformed and failing evidence first;
- accepted-language coverage grows behind that shield.

Presenter script: Read aloud:

- soundness bugs outrank parser gaps; kernel-adjacent tests emphasize malformed and failing evidence first;
- accepted-language coverage grows behind that shield.

After the example, say which design obligation the syntax makes visible.

Key Points:

- bilingual language specification: 24 chapters plus appendices, English canonical with Japanese companions (`doc/spec/`);
- architecture specification: 24 documents covering the pipeline, kernel, certificate format, and AI agent interface (`doc/design/architecture/`);
- a Rust workspace of 20 crates - lexer, parser, resolver, checker, VC generator, ATP bridge, kernel, build system - roughly 400k lines including tests;
- focused audits completed in 2026: kernel soundness, template logic encoding, SAT solver dependency;
- the roadmap is decomposed into small, independently verifiable tasks.

Message:

- The end-of-2026 alpha is a milestone on a running track, not a promise on an empty page.

└ Part 11. Roadmap And Collaboration

└ 11.1 - Where The Project Stands Today

11.1 - Where The Project Stands Today

Key Points:

- **bilingual language specification:** 24 chapters plus appendices, English canonical with Japanese companions (`doc/spec/`);
- **architecture specification:** 24 documents covering the pipeline, kernel, certificate format, and AI agent interface (`doc/design/architecture/`);
- **a Rust workspace of 20 crates** - lexer, parser, resolver, checker, VC generator, ATP bridge, kernel, build system - roughly 400k lines including tests;
- **focused audits completed in 2026:** kernel soundness, template logic encoding, SAT solver dependency;
- **the roadmap is decomposed into small, independently verifiable tasks.**

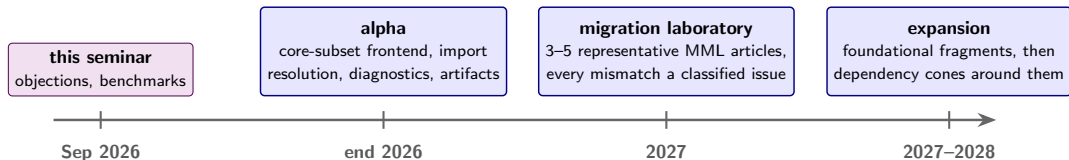
Message:

- The end-of-2026 alpha is a **milestone on a racing track, not a promise on a empty page.**

Presenter script: Read aloud:

- bilingual language specification: 24 chapters plus appendices, English canonical with Japanese companions (`doc/spec/`);
- architecture specification: 24 documents covering the pipeline, kernel, certificate format, and AI agent interface (`doc/design/architecture/`);
- a Rust workspace of 20 crates - lexer, parser, resolver, checker, VC generator, ATP bridge, kernel, build system - roughly 400k lines including tests;
- focused audits completed in 2026: kernel soundness, template logic encoding, SAT solver dependency;
- the roadmap is decomposed into small, independently verifiable tasks.

11.2 - Migration Is A Research Program



Phases:

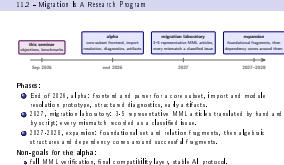
- 1 End of 2026, alpha: frontend and parser for a core subset, import and module resolution prototype, structured diagnostics, early artifacts.
- 2 2027, migration laboratory: 3-5 representative MML articles translated by hand and by script; every mismatch recorded as a classified issue.
- 3 2027-2028, expansion: foundational set and relation fragments, then algebraic structures and dependency cones around successful fragments.

Non-goals for the alpha:

- full MML verification, final compatibility layer, stable AI protocol.

Part 11. Roadmap And Collaboration

11.2 - Migration Is A Research Program



Presenter script: Read aloud:

- End of 2026, alpha: frontend and parser for a core subset, import and module resolution prototype, structured diagnostics, early artifacts.
- 2027, migration laboratory: 3-5 representative MML articles translated by hand and by script; every mismatch recorded as a classified issue.
- 2027-2028, expansion: foundational set and relation fragments, then algebraic structures and dependency cones around successful fragments.
- full MML verification, final compatibility layer, stable AI protocol.

Key Points:

- translated articles and lines; accepted parser subset;
- resolved imports versus unresolved dependencies;
- obligations closed deterministically versus by ATP-plus-certificate;
- memory and wall-clock per module, incremental versus clean;
- compatibility decisions that required human judgment.

- translated articles and lines; accepted parser subset;
- resolved imports versus unresolved dependencies;
- obligations closed deterministically versus by ATP-plus-certificate;
- memory and wall-clock per module, incremental versus clean;
- compatibility decisions that required human judgment.

Presenter script: Read aloud:

- translated articles and lines; accepted parser subset;
- resolved imports versus unresolved dependencies;
- obligations closed deterministically versus by ATP-plus-certificate;
- memory and wall-clock per module, incremental versus clean;
- compatibility decisions that required human judgment.

Policy:

- preserve theorem identity through origin metadata;
- keep compatibility aliases where they help migration;
- record every divergence from old behavior with a reason and a test.

Risk	Mitigation
compatibility work consumes the project registrations behave differently package layout breaks journal links AI edits hide migration mistakes	representative slices first, no big-bang translation trace artifacts and comparison reports early origin metadata, article-to-library identifiers Red edits stay forbidden; verifier artifacts required

11.4 - Compatibility Policy And Risks	
deep dive	
Policy: • preserve theorem identity through origin metadata; • keep compatibility aliases where they help migration; • record every divergence from old behavior with a reason and a test.	
Risk	Mitigation
unpublish work to remove the project	representative definition, ex. library, standards
significance between different	trace aliases and compatibility reports work
package layer: breaks parallelism	origin metadata, article/library identifier
Alias/alias migration mistakes	Post-note way forbidden; neither aliases required

Presenter script: Read aloud:

- preserve theorem identity through origin metadata;
- keep compatibility aliases where they help migration;
- record every divergence from old behavior with a reason and a test.

For the table, read the row labels first, then the design reason.

Key Points:

- a prioritized list of migration benchmark articles;
- agreement on compatibility metadata needs;
- review notes on the eight stories, especially structures and clusters;
- a first paper outline;
- a shared list of objections and risks.

└ Part 11. Roadmap And Collaboration

└ 11.5 - What September 2026 Should Produce

11.5 - What September 2026 Should Produce

Key Points:

- a prioritized list of migration benchmark articles;
- agreement on compatibility metadata needs;
- review notes on the eight stories, especially structures and clusters;
- a first paper outline;
- a shared list of objections and risks.

Presenter script: Read aloud:

- a prioritized list of migration benchmark articles;
- agreement on compatibility metadata needs;
- review notes on the eight stories, especially structures and clusters;
- a first paper outline;
- a shared list of objections and risks.

Questions:

- Which MML articles are small but structurally representative?
- Which idioms are culturally essential, beyond technical convenience?
- Which of the eight stories is most wrong, and why?
- What migration result would convince the community that Evo is serious?

Questions:

- Which MML articles are small but structurally representative?
- Which idiom is culturally essential, beyond technical convenience?
- Which of the eight stories is most wrong, and why?
- What migration result would convince the community that Evo is serious?

Presenter script: Read aloud:

- Which MML articles are small but structurally representative?
- Which idioms are culturally essential, beyond technical convenience?
- Which of the eight stories is most wrong, and why?
- What migration result would convince the community that Evo is serious?

Slide question:

What must Mizar Evo preserve so that the Mizar community still recognizes it as Mizar?

Presenter script: Say:

- Return to the opening example: the structure that still reads as Mizar.
- Invite disagreement story by story, not only in general terms.

Final slide:

*Mizar Evo should be modern where scale demands it,
and conservative where Mizar's mathematical identity depends on it.*

Presenter script: After the example, say which design obligation the syntax makes visible. Use this as the transition: Closing. Point to the visible example, question, or diagram before moving on.

Prepared short excerpts for Beamer conversion:

Purpose	Source	Lines	Used in
Structure definition	algstr_0.miz	37-40	Frames 0.2, 3.1
Article environment	algstr_0.miz	15-25	Frame 2.1
Registration/cluster	algstr_0.miz	104-109	Frame 4.1

Prepared short excerpts for Beamer conversion:

Purpose	Source	Lines	Used in
Proof citation	struct_0.miz	637-643	Frame 5.4
Induction scheme	nat_1.miz	near 90	Frame 7.1

Source URLs:

- ALGSTR_0: <https://mizar.uwb.edu.pl/version/current/mml/algstr_0.miz>
- STRUCT_0: <https://mizar.uwb.edu.pl/version/current/mml/struct_0.miz>
- NAT_1: <https://mizar.uwb.edu.pl/version/current/mml/nat_1.miz>

Mizar Evo

└ Backup A. Prepared Exact Examples

└ Backup A. Prepared Exact Examples (1/2)

Prepared short excerpts for Beamer conversion:

Excerpt	Source	Lines	Used in
Signature definition	algstr_0.miz	1548	Frames 6,7,3,1
Addition introduction	algstr_0.miz	1563	Frame 2,1
Regularity lemma	algstr_0.miz	1864-88	Frame 6,1

Prepared short excerpts for Beamer conversion:

Excerpt	Source	Lines	Used in
Proof sketch	struct_0.miz	832-843	Frame 5,4
Induction scheme	nat_1.miz	lines 88	Frame 7,1

Source URLs:

- ALGSTR_0: <https://mizar.uwb.edu.pl/version/current/mml/algstr_0.miz>
- STRUCT_0: <https://mizar.uwb.edu.pl/version/current/mml/struct_0.miz>
- NAT_1: <https://mizar.uwb.edu.pl/version/current/mml/nat_1.miz>

Presenter script: Read aloud:

- ALGSTR_0: <https://mizar.uwb.edu.pl/version/current/mml/algstr_0.miz>
- STRUCT_0: <https://mizar.uwb.edu.pl/version/current/mml/struct_0.miz>
- NAT_1: <https://mizar.uwb.edu.pl/version/current/mml/nat_1.miz>

For the table, read the row labels first, then the design reason.

Attribution note:

- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.

Presenter script: Read aloud:

- MML plain-text files state GPL-3.0-or-later / CC-BY-SA-3.0-or-later terms; keep article attribution, URLs, and line numbers in speaker notes.

The slides show examples only. The authoritative grammar and semantics live in the specification; this map replaces the EBNF that earlier drafts put on slides.

Topic	Specification source (under doc/spec/en/)
Modules and imports	12.modules_and_namespaces.md
Structures and inheritance	05.structures.md
Attributes and modes	06.attributes.md, 07.modes.md
Registrations and reductions	17.clusters_and_registrations.md
Templates and schemes	18.templates.md
Algorithms and MVM	20.algorithm_and_verification.md
Packages and artifacts	23.package_management_and_build_system.md

Mizar Evo

└ Backup B. Specification Reference Map

└ Backup B. Specification Reference Map (1/2)

The slides show examples only. The authoritative grammar and semantics live in the specification; this map replaces the EBNF that earlier drafts put on slides.

Topic	Specification source (under doc/spec/sem/)
Modules and imports	00.modules_and_imports.mf
Structure and inheritance	01.structure.mf
Attributes and modes	04.attributes.mf, 07.modes.mf
Engines and reductions	07.engines_and_reduction.mf
Templates and schemas	08.templates.mf
Algorithms and verification	20.algorithms_and_verification.mf
Packages and macros	23.packages_and_macros_and_utilities.mf

Presenter script: Read aloud:

- The slides show examples only. The authoritative grammar and semantics live
- in the specification; this map replaces the EBNF that earlier drafts put on
- slides.

For the table, read the row labels first, then the design reason.

Backup B. Specification Reference Map (2/2)

Topic	Specification source (under doc/spec/en/)
Cross-chapter grammar	appendix_a.grammar_summary.md
Worked library sketches	sample_codes.md

└ Backup B. Specification Reference Map

└ Backup B. Specification Reference Map (2/2)

Topic	Specification source (under doc/spec/evm/)
Crashmap in grammar	crashmap_v4_crashmap_v4_syntax.m
Worked history matches	evm_v4_crash.m

Presenter script: For the table, read the row labels first, then the design reason.

Use this as the transition: Backup B. Specification Reference Map (2/2). Point to the visible example, question, or diagram before moving on.

Required diagrams for the final deck (figures/*.tex, TikZ standalone;
build each with pdflatex inside figures/):

- ① Three pressures on a proven design (Part 1). [done: figures/three_pressures.pdf, used in frame 1.5]
- ② Environment-to-import migration (story 1). [done: figures/envIRON_migration.pdf, used in frame 2.5]
- ③ Structure inheritance and diamond coherence (story 2). [done: figures/diamond_inheritance.pdf, used in frame 3.5]
- ④ Reasoning boundary: semantics / ATP search / kernel checking (story 4). [done: figures/reasoning_boundary.pdf, used in frame 5.2]
- ⑤ Certificate object and replay (story 4). [done: figures/certificate_replay.pdf, used in frame 5.3]

Backup C. Diagram List (1/2)

Required diagrams for the final deck (figures/*.tex, TikZ standalone;
build each with pdflatex inside figures/):

- Three pressures on a proven design (Part 1). [done: figures/three_pressures.pdf, used in frame 1.5]
- Environment-to-import migration (story 1). [done: figures/environs_migration.pdf, used in frame 2.5]
- Structure inheritance and diamond coherence (story 2). [done: figures/diamond_inheritance.pdf, used in frame 3.5]
- Reasoning boundary: semantics / ATP search / kernel checking (story 4). [done: figures/reasoning_boundary.pdf, used in frame 5.2]
- Certificate object display (story 4). [done: figures/certificate_display.pdf, used in frame 5.3]

Presenter script: Read aloud:

- Required diagrams for the final deck (figures/*.tex, TikZ standalone;
- Three pressures on a proven design (Part 1). [done: figures/three_pressures.pdf, used in frame 1.5]
- Environment-to-import migration (story 1). [done: figures/environs_migration.pdf, used in frame 2.5]
- Structure inheritance and diamond coherence (story 2). [done: figures/diamond_inheritance.pdf, used in frame 3.5]
- Reasoning boundary: semantics / ATP search / kernel checking (story 4). [done: figures/reasoning_boundary.pdf, used in frame 5.2]

build each with pdflatex inside figures/):

- ➊ Incremental fingerprint graph (story 5). [done: figures/fingerprint_graph.pdf, used in frame 6.2]
- ➋ Formalized Mathematics article-to-library link model (story 8). [done: figures/fm_links.pdf, used in frame 9.2]
- ➌ Full pipeline with story overlay (Part 10). [done: figures/pipeline.pdf, used in frame 10.1]
- ➍ Roadmap timeline (Part 11). [done: figures/roadmap_timeline.pdf, used in frame 11.1]

```
build each with pdflatex inside figures/):
```

- Incremental fingerprint graph (story 5). [done: figures/fingerprint_graph.pdf, used in frame 6.2]
- Formalized Mathematics article-to-library link model (story 8). [done: figures/fm_links.pdf, used in frame 9.2]
- Full pipeline with story overlay (Part 10). [done: figures/pipeline.pdf, used in frame 10.1]
- Roadmap timeline (Part 11). [done: figures/roadmap_timeline.pdf, used in frame 11.1]

Presenter script: Read aloud:

- Incremental fingerprint graph (story 5). [done: figures/fingerprint_graph.pdf, used in frame 6.2]
- Formalized Mathematics article-to-library link model (story 8). [done: figures/fm_links.pdf, used in frame 9.2]
- Full pipeline with story overlay (Part 10). [done: figures/pipeline.pdf, used in frame 10.1]
- Roadmap timeline (Part 11). [done: figures/roadmap_timeline.pdf, used in frame 11.1]

Possible paper title:

Mizar Evo: Readable, AI-Ready, and Scalable Formal Mathematics

Possible sections:

- 1 Introduction: why Mizar needs evolution now.
- 2 Mizar as baseline: readability, MML, Formalized Mathematics.
- 3 Design principles and the three pillars.
- 4 Language evolution: dependencies, structures, registrations, templates.
- 5 Verifier architecture, certificates, and the small kernel.
- 6 Verified computation and the MVM.
- 7 AI-safe proof development.

Mizar Evo

└ Backup D. Paper Outline Seed

└ Backup D. Paper Outline Seed (1/2)

Possible paper title:

Mizar Evo: Readable, Abstract, and Scalable Formal Mathematics

Possible sections:

- 1 Introduction: why Mizar needs evolution now.
- 2 Mizar as baseline: readability, MML, Formalized Mathematics.
- 3 Design principles and the three pillars.
- 4 Language evolution: dependencies, abstractions, rigour, wisdom, templates.
- 5 Verified architecture, certificates, and the small kernel.
- 6 Verified computation and the MVM.
- 7 Abstract proof development.

Presenter script: Read aloud:

- Introduction: why Mizar needs evolution now.
- Mizar as baseline: readability, MML, Formalized Mathematics.
- Design principles and the three pillars.

After the example, say which design obligation the syntax makes visible.

Possible sections:

- 1 Package-based library and publication workflow.
- 2 Migration plan and evaluation metrics.
- 3 Related work and collaboration agenda.

Possible sections:

- Package-based library and publication workflow.
- Migration plan and evaluation metrics.
- Related work and collaboration agenda.

Presenter script: Read aloud:

- Package-based library and publication workflow.
- Migration plan and evaluation metrics.
- Related work and collaboration agenda.

Use this checklist before converting to Beamer:

- Does every story open with a real cost, not a feature announcement?
- Does every criticism acknowledge why the current practice was useful?
- Does every code example carry its status label (exact MML excerpt, specification example, or sketch)?
- Is every exact excerpt attributed with article and line numbers?
- Does every story end with questions the audience can actually answer?
- Are Red AI edits clearly forbidden?
- Are migration claims measurable?

Use this checklist before converting to Beamer:

- Is EBNF absent from all frames?

Use this checklist before convening to Beamer:

- Does every story open with a real cost, not a feature announcement?
- Does every criticism acknowledge why the current practice was useful?
- Does every code example carry its status label (exact MML excerpt, a specification example, or a sketch)?
- Is every exact excerpt attributed with a title and line numbers?
- Does every story end with questions the audience can actually answer?
- Are Red Alerts clearly justified?
- Are migration claims measurable?

Use this checklist before convening to Beamer:

- Is EBNF a best from all formats?

Presenter script: Read aloud:

- Does every story open with a real cost, not a feature announcement?
- Does every criticism acknowledge why the current practice was useful?
- Does every code example carry its status label (exact MML excerpt, specification example, or sketch)?
- Is every exact excerpt attributed with article and line numbers?
- Does every story end with questions the audience can actually answer?

Prepared answers for objections the stories do not raise themselves:

Objection	Prepared answer
Why not improve current Mizar incrementally?	The pains are boundary-shaped: article granularity, monolithic trust, implicit environments. Boundaries cannot be moved incrementally, but everything above them is deliberately conservative.
What happens to authorship and credit of MML articles?	Origin metadata preserves article identity and authorship through migration; the pub namespace keeps published articles frozen and citable.
Is this a fork of the community?	It is a proposal to the community; this visit is its first review. The namespace governance model assumes the Mizar team governs the <code>mml</code> root.

Mizar Evo

└ Backup F. Anticipated Objections

└ Backup F. Anticipated Objections (1/2)

Prepared answers for objections the stories do not raise themselves:

Objection	Prepared answer
Why not improve current Mizar incrementally?	The pairs are hard to shape: article granularities, meaningful ones, implicit existentials. Incrementation cannot be mixed incrementally, but everything above them is de facto article construction.
What happens to authorship and credit of MML articles?	Origin meta-data promotes article identity and authorship through signatures; the journaling and simple publication articles to news and details.
Is this a test of the community?	It is a proposal to the community; this idea is in the realm. The newspaper governance model assumes the Mizar team generates the content.

Presenter script: Say:

- Use these only if raised; do not present them proactively.

For the table, read the row labels first, then the design reason.

Prepared answers for objections the stories do not raise themselves:

Objection	Prepared answer
What about GPL / CC-BY-SA obligations?	Migration preserves license and attribution of MML content; toolchain licensing is open for discussion.
Is the AI angle hype?	AI assistance is an optional layer that never enters the trusted base; every proposal stands without it.
Why a new kernel instead of the existing checker?	Certificate replay needs a small, auditable core. The existing checker's semantics remain the reference for obligation generation.

└ Backup F. Anticipated Objections

└ Backup F. Anticipated Objections (2/2)

Prepared answers for objections the stories do not raise themselves:

Objections	Prepared answer
What about GPL / CC-BY-SA violations?	Mizarin promotes license and attribution of MM1 content; redistribute licensing is open for discussion. All evidence is on original paper that never enters the trained language program words without it.
Is the AI single loop?	Conscious reply sends a small, as that is core. The editing checker's answer determines the response for all future generations.

Presenter script: For the table, read the row labels first, then the design reason.

Use this as the transition: Backup F. Anticipated Objections (2/2). Point to the visible example, question, or diagram before moving on.